NTU-114-1-ADL-HW2

Chia-Chun Liang

Data Science Degree Program

National Taiwan University

Student ID: d14948004

Email: d14948004@ntu.edu.tw

Oct 2025

Task Description and Overview

The objective of this assignment is to do machine translation between *classical Chinese* and *modern Chinese* which is also shown in Figure 1.

图 1: An overview of the whole task process.

Report - Q1: LLM Tuning

Describe

Training dataset (2%) The training dataset consists of:

- **Training samples**: 10,000 examples (train.json)
- **Evaluation samples**: 250 examples (public_test.json)
- **Data format**: JSON format containing instruction-output pairs. The following is the demo of some training data:

```
[
  {
    "id": "b6fe97e8-1c24-47a5-a399-48f9fa6f41cc",
    "instruction": "翻譯成現代文: \n河陰之役, 故得免害。\n答案: ",
    "output": "在河陰的變亂中, 因此得以避免受害。"
  },
  {
    "id": "25e080dd-9019-4eba-9ca3-131bc4b73653",
    "instruction": "自從遷都許之後, 曹操掌權, 天子自顧不暇, 百官都是充數而已。\n翻譯成文言文: ",
    "output": "自都許之後, 權歸曹氏, 天子總己, 百官備員而已。"
  }
]
```

How to tune model (2%) I used Unsloth to fine-tune the model, which provides a more efficient way to conduct parameter-efficient fine-tuning. *Unsloth* is an optimization library that accelerates the training process by up to 2x while reducing memory usage by 30% on `Qwen2.5`. (Detail in this colab.) Unsloth also provide the optimized model of `Qwen3-4B` with its official weight.

Initial Approach and Problem in Data preprocessing: At the beginning, I used a standard datapreprocessing with Alpaca-form dataset where the entire sequence (both instruction and output) contributed to the loss calculation.

It did confuse me a lot is that although the training loss decreased significantly during training, the model's actual performance was very poor (mean perplexity is around 98 to 99). The model failed to generate correct translations and often repeated the input or produced irrelevant outputs.

Root Cause Analysis: The issue was that the model was learning patterns from both *instruction* and *answer* (output) portions. Since the similarity of the given prompt(instruction), the model to memorize the input format rather than learning the actual translation task.

Solution - Output-Only Loss Calculation: I discovered that the loss should only be calculated on the output portion, while the instruction portion should be masked. This is implemented by:

- Setting labels to -100 for all instruction tokens (including system prompt and question)
- Only calculating loss on the actual translation output tokens
- Appending EOS token at the end for proper sequence termination

The data preprocessing pipeline includes the following steps:

1. **Custom Prompt Engineering**: At first I designed the prompt to be very long, but I found that this would cause errors (`RuntimeError: Output 0 of UnslothFusedLossBackward`). Later I changed it to be shorter and it worked. The final system prompt was designed for Classical Chinese (文言文) to Modern Chinese (白話文) translation is shown in the following:

```
SYS_PROMPT  = "你是一個中文語言專家，你將進行文言文與白話文之間的精準翻譯。\
翻譯時，請產生最符合歷史語言習慣的中文輸出；\
請注意，只能使用繁體中文；請在回答時保持簡潔，不需要任何其他回饋。以下是你的任務"
```

图 2: The designed prompt in training

2. **Label Masking**: Only the output portion is used for loss calculation, while the instruction portion is masked with -100 labels to prevent the model from learning from the question part.
3. **Token Processing**:
   - Question tokens are tokenized with special tokens

- Answer tokens are tokenized without additional special tokens
- EOS token is appended at the end of each sequence

4. **Sequence Length Filtering**: Sequences longer than 1999 tokens were filtered out to ensure stable training (max_seq_length = 2048).

The main python code for the entire data processing is shown below:

```
for instruction, output in zip(instructions, outputs):

    # Seperate 'Question' and 'Answer'
    question_text = SYS_PROMPT+instruction
    answer_text = output + EOS_TOKEN

    # Tokenize 'Question' and 'Answer' respectively
    question_tokens = tokenizer(
        question_text,
        add_special_tokens=True,  # ADD BOS token
        truncation=False,
        padding=False,
    )

    answer_tokens = tokenizer(
        answer_text,
        add_special_tokens=False,  # W/O Special token
        truncation=False,
        padding=False,
    )

    # Combine input_ids
    input_ids = question_tokens["input_ids"] + answer_tokens["input_ids"]

    # Create 'labels': -100 for question, real token ids for answer
    labels = (
        [-100] * len(question_tokens["input_ids"]) +  # Excluded loss in question
        answer_tokens["input_ids"]                     # Consider loss in answer
    )
```

Hyper-parameters (2%) The model was fine-tuned using QLoRA (Quantized Low-Rank Adaptation) with the following configuration:

表 1: LoRA Configuration

Parameter	**Value**
Rank (r)	32
LoRA Alpha	64
LoRA Dropout	0 (optimized)
Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj
Bias	none
Task Type	Causal LM

Training Strategy The fine-tuning process leverages Unsloth's optimized implementation, which provides several key advantages:

1. **Efficient Memory Management**:
   - Unsloth's smart gradient offloading automatically manages VRAM usage
   - 4-bit quantization with bitsandbytes (NF4 format) reduces model size
   - Dynamic batch size handling prevents out-of-memory errors
2. **Accelerated Training**:
   - Unsloth patches 36 layers with optimized QKV and MLP operations
   - Fast model loading with automatic detection of optimal settings
   - Native 2x faster inference capability for evaluation
3. **Optimization Configuration**:
   - Optimizer: AdamW (torch implementation)
   - Weight decay: 0.01
   - Learning rate scheduler: Cosine annealing with warmup
   - Warmup strategy: 5 steps with 0.03 warmup ratio
4. **Loss Calculation Strategy**:
   - Only answer tokens contribute to loss computation
   - Instruction tokens are masked with label = -100
   - Custom `DataCollatorForSeq2Seq` ensures proper padding and masking
   - This approach prevents the model from learning question patterns

5. **Gradient Management**:
   - Gradient accumulation steps: 4 (effective batch size = 8)
   - Gradient checkpointing: `Disabled` for avoiding `RuntimeError: Output 0 of UnslothFusedLossBackward`.
   - Unsloth's automatic gradient offloading enabled for VRAM optimization
6. **Model Saving and Evaluation**:
   - Evaluation performed every 50 steps on validation set
   - Best model selected based on *lowest validation loss*
   - Total Steps: 2000 (1.6 epochs)
   - Load best model at end: True (checkpoint-1250 selected)
   - Only top 3 checkpoints retained to save disk space

Implementation Details: The Unsloth library was integrated using the `FastLanguageModel` API, which automatically applies optimizations including:

- Automatic detection of optimal data types (FP16/BF16)
- Efficient RoPE scaling for extended context lengths
- Optimized attention mechanisms (eager attention for compatibility)
- Streamlined PEFT integration for LoRA adapters

Show your performance

Final performance (2%) The final model with the lowest validation loss is checkpoint-1250.

The used inference prompt is shown as Figure 3

```python
def get_prompt(instruction: str) -> str:
    sys_prompt = "你是一個專精中國古文（文言文）與現代中文的語言專家，\
                  擅長文言文與白話文之間的準確翻譯與風格轉換。\
                  你的任務將會是`將中文古文（文言文）精準翻譯為中文白話文`，\
                  或是`將中文白話文精準翻譯為中文古文（文言文）`。\
                  要求：\
                  - 翻譯時，請產生最符合歷史語言習慣的中文輸出\
                  - 請注意，只能使用 ** 繁體中文 ** 。\
                  - 請檢查所有資訊是否準確，並在回答時保持簡潔，不需要任何其他回饋。\
                  - 若任務開頭有「文言文翻譯：」，請以「答案：」開頭，並只給相對應的正確回答\
                  以下是你本次的任務：\n\n "
    return sys_prompt+instruction
```

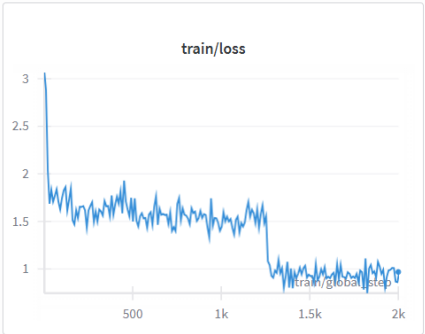
图 3: The designed prompt in inference

The evaluation code is following by the below instruction, and the testdata is used the given *data/public_test.json*:

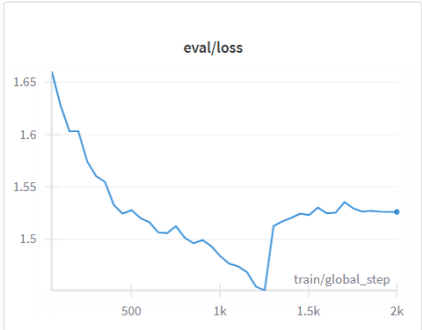
```
!python ppl.py --base_model_path "Qwen/Qwen3-4B" \
               --peft_path /path/to/checkpoint-1250 \
               --test_data_path /path/to/testdata
```

After run the given script `ppl.py`, it turned out that mean perplexity **6.346625**.

Plot the learning curve on the public testing set(2%) As shown in Figure 4, both training and evaluation losses decrease rapidly in the early stage, suggesting that the model quickly learns general patterns from the data. However, around one specific epoch, there is a noticeable gap between the two curves: the **training loss drops sharply**, while the **evaluation loss slightly rises**. This divergence indicates that the model starts to overfit the training data. It tells us the strategy to increase training epoch may fail to reach a better performance.



(a) Training loss during 2000-step training



(b) Eval loss during 2000-step training

图 4: Training and evaluation loss curves during 2000-step training.

Report - Q2: LLM Inference Strategies

Zero-Shot

What is your setting? How did you design your prompt? (1%) I referred to this repository for designing the prompt, which includes (1) role definition and (2) task specification. Additionally, since our context is (3) Traditional Chinese, the prompt explicitly specifies this requirement.

Furthermore, because the dataset instances are not in a QA-style format (unlike a conversational assistant that might respond with "Okay, here is ⋯"), but instead require a *direct translation* of the corresponding Classical or Modern Chinese sentence,

we added instructions to ensure the model outputs only the required sentence without extra commentary.

Finally, we noted that instances starting with “文言文翻譯：” correspond to outputs beginning with “答案：”; therefore, we included this rule in the prompt as well.

The final prompt for zero-shot is as Figure 5:

```
def get_zero_shot_prompt(instruction: str) -> str:
    sys_prompt = "你是一個專精中國古文（文言文）與現代中文的語言專家，\
                  擅長文言文與白話文之間的準確翻譯與風格轉換。\
                  你的任務將會是`將中文古文（文言文）精準翻譯為中文白話文`，\
                  或是`將中文白話文精準翻譯為中文古文（文言文）`。\
                  要求：\
                  - 翻譯時，請產生最符合歷史語言習慣的中文輸出\
                  - 請注意，只能使用 ** 繁體中文 ** 。\
                  - 請檢查所有資訊是否準確，並在回答時保持簡潔，不需要任何其他回覆。\
                  - 若任務開頭有「文言文翻譯：」，請以「答案：」開頭，並只給相對應的正確回答\
                  以下是你本次的任務：\n\n "
    return sys_prompt+instruction
```

图 5: The designed prompt for zero-shot

Few-Shot (In-Context Learning)

What is your setting? How did you design your prompt? (1%) I followed the same design as in *Zero-Shot*, but added three in-context examples to the prompt.

How many in-context examples are utilized? How did you select them? (1%) Three examples were selected from `train.json`:

1. A translation from Modern Chinese to Classical Chinese.
2. A translation from Classical Chinese to Modern Chinese, demonstrating the “文言文翻譯:” → “答案:” format.
3. Another variation of Classical-to-Modern translation using alternative expressions.

The final prompt for few-shot is as Figure 6:

Comparison

What’ s the difference between the results of zero-shot, few-shot, and LoRA? (2%) For both *Zero-Shot* and *Few-Shot*, I did not modify the parameters of `Qwen/Qwen3-4B`, but the prompts had to be longer and more detailed to guide the model effectively. In contrast, *LoRA* fine-tunes the model on task-specific data, leading to much lower perplexity.

```python
def get_few_shot_prompt(instruction: str) -> str:
    sys_prompt = "你是一個專精中國古文（文言文）與現代中文的語言專家，\
                  擅長文言文與白話文之間的準確翻譯與風格轉換。\
                  你的任務將會是`將中文古文（文言文）精準翻譯為中文白話文`，\
                  或是`將中文白話文精準翻譯為中文古文（文言文）`。\
                  要求：\
                  - 翻譯時，請產生最符合歷史語言習慣的中文輸出\
                  - 請注意，只能使用 ** 繁體中文 ** 。\
                  - 請檢查所有資訊是否準確，並在回答時保持簡潔，不需要任何其他回饋。\
                  - 若任務開頭有「文言文翻譯：」，請以「答案：」開頭，並只給相對應的正確回答\
                  \n\n以下是第一個範例：\
                  - 沒過十天，鮑泉果然被拘捕。\n幫我把這句話翻譯成文言文\
                  - 後未旬，果見囚執。\
                  \n\n以下是第二個範例：\
                  - 文言文翻譯：\n明日，趙用賢疏入。\
                  - 答案：第二天，趙用賢的疏奏上。\
                  \n\n以下是第三個範例：\
                  - 祀明堂，遷給事中兼龍圖閣學士。\n翻譯成現代文：\
                  - 祭祀明堂，升任給事中，兼龍圖閣學士。\
                  以下是你本次的任務：\n\n "
    return sys_prompt+instruction
```

图 6: The designed prompt for few-shot

The following table shows the evaluation results in terms of perplexity (lower is better):

Setting	**Perplexity (Lower is Better)**
Zero-Shot	619.29
Few-Shot (In-Context)	361.39
LoRA	6.35

表 2: Perplexity comparison across different methods.

These results demonstrate a significant improvement after fine-tuning with LoRA compared to pure inference strategies.

Report - Q3: Bonus –Llama3-Taiwan (8B)

I fine-tuned the Llama3-Taiwan (8B) model using the same hyperparameters and training setup as our above parameter setting:

- **Gradient accumulation steps:** 4
- **Warmup steps:** 5
- **Warmup ratio:** 0.03
- **Optimizer:** AdamW (`adamw_torch`, suitable for QLoRA)
- **Weight decay:** 0.01

The validation loss achieved during fine-tuning was **1.763639**. Based on this, the computed perplexity on the validation set was **9.85465625**.

Acknowledgement

I used ChatGPT, Claude and Gemini to assist with parameter tuning, code development, and report refinement.

However, issues such as incorrect data processing and the RuntimeError caused by zero ‘UnslothFusedLossBackward‘ due to an overly long system prompt were identified and resolved independently. This experience highlighted the importance of careful debugging and critical thinking, although AI tools were still very helpful throughout the process.